Luma Development Reference

Table of contents

About Luma

Land Use Mapping for All (Luma) is an online mapping platform that integrates open-source spatial data and cloud-based computing for generating land use and land cover (LULC) maps. Luma is designed to accommodate a wide range of users through its user-friendly interface and guided steps, enabling anyone to easily produce custom LULC maps tailored to their needs.

This work is part of the **Evolving Participatory Information System for Nature-based Climate Solutions (Epistem)** initiative, which aims to develop an open-source landscape monitoring technology that can address multiple thematic requirements of diverse actors and stakeholders of nature-based climate solutions.

Collaborative Development Guideline

About the Three-Stream Approach

- **What is the three-stream approach?**

The three-stream approach is a workflow that separates brainstorming, design, and implementation into clear yet connected streams. Each stage of development has its own dedicated space, allowing for a structured and transparent decision-making and implementation process throughout the development cycle instead of mixing ideas, references, and implementations in scattered places.

- **Why the three-stream approach?**

The approach intends to solve a common problem in collaborative software development: ideas were discussed in one place, decisions were written somewhere else, and code evolved on its own, making it hard to tell what was current or why something changed. Someone working a week later or in a different time zone would often re-solve the same problem or unknowingly work against an outdated assumption.

The three-stream approach keeps context visible through asynchronous communication, so work stays aligned even when people are not in the same place or working at the same time.

- **What are the three streams?**
  - The "Braindump" (on Notion)

    This is where ideas start. We use this Braindump to record all notes and discussions regarding future developments and improvements of the Luma Geospatial Engine. Nothing here is final and things are allowed to change or be discarded.
  - The "Development Reference" (using Quarto)

    This is where ideas become decisions. Content here is structured and stable enough for developers to refer to. It provides shared context so contributors can understand past decisions, even if they were not part of the original discussion.
  - The "Code Implementation" (Luma Geospatial Engine Python package)

    The Luma Geospatial Engine is the final implementation of the development, in the form of a Python-based package that powers the Luma platform.

Both the Development Reference and Code Implementation are hosted on GitHub for version control.

Steps of the Three-Stream Approach

1. Start by writing your comments in the **Braindump** under the relevant section. Tag the person you want to discuss with so your input is visible and does not get missed.
    - If someone disagrees with your comment, add the topic to the **To-Settle** list. Use the page to write down the key arguments so they can be discussed efficiently in a meeting. Once the disagreement is resolved, move the topic to **Work in Progress**.
    - If there is no disagreement, you can add the topic directly to **Work in Progress**.
2. For agreed development work, create a new branch in **luma-dr** and document the proposed changes there.
3. Next, open a pull request in **luma-dr** from your new branch to the main branch. **Pull requests are reviewed during regular pull request meetings, where changes are either accepted or rejected.**
4. Once changes are merged into the main branch of **luma-dr**, contributors can begin implementing them in **luma-stack**. Implementation should always start from a new branch.
5. Submit a pull request on **luma-stack** to propose your changes for review at the next pull request meeting. If needed, also submit a pull request on **luma-dr** to update the documentation describing the changes.
6. During each pull request meeting, the team will:
    - Review and accept pull requests in **luma-stack** that implement previously approved changes
    - Update **luma-dr** based on approved pull requests
    - Review new pull requests in **luma-dr** proposed for future updates

> Tip
>
> For more reference on the workflow (including a case example), refer to this slide presented during the Bootcamp.

The "Braindump"

The Braindump serves as the central hub for refining ideas and discussions related to Luma's development prior to their formal inclusion in the Development Reference. It contains organized pages that ensure alignment with the current Development Reference, along with sections dedicated to other discussion topics.

- **Structured pages for Luma Geospatial Engine development**
  - The content of each page in this section reflects the latest version of the Development Reference (i.e., the `main` branch of luma-dr).
  - Ideas and discussions can be provided by commenting on the related section of each page. **Tag the relevant person in the comment to ensure the comment is read by others.**
  - The comments will be resolved in two cases:
    * No follow-up is needed after the discussion.
    * The follow-up has been listed in the **Work in Progress** or **To-Settle** section.
- **Structured pages for Luma User Interface development**

  This section is a database dedicated to the implementation details of the Luma User Interface.
- **To-Settle**
  - This section is dedicated to listing topics that arise from disagreements among multiple people in the comments on the **Structured pages for Luma Geospatial Engine development** and that require further discussion or a decision before updates are documented in the Development Reference.
  - The disagreement should be briefly described inside the page.
  - Choose an urgency level of the conflict to determine if the meeting should be scheduled as soon as possible.
  - Once the conflict is settled, the issue can either be moved to be listed in the **Work in Progress** or dismissed.
- **Work in Progress**
  - This section is dedicated for tracking ongoing tasks related to Luma Geospatial Engine updates.
  - Each task goes through several stages of development according to the contribution workflow. The stage at which an issue is currently is noted in the **Completion status** column.

- **Unresolved issues from Pull Request approvals**

  This section records issues or feedback that emerged during the pull request review process and need additional work or clarification before approval.

- **Meeting Notes**

  A collection of notes, summaries, and key decisions from meetings related to Luma's development activities.

- **Sketchbooks**

  A space for "sketching out" ideas or conceptual designs before formal documentation. This section is intentionally unstructured to encourage open exploration.

Version Control

Version control is a core component of the Luma ecosystem and is managed through GitHub, which serves as the central platform for tracking both Quarto based Development Reference and Luma Geospatial Engine code implementation. GitHub provides a transparent and traceable environment for managing changes to code and documentation over time. Contributors create a dedicated branch for each feature, fix, or documentation update. Once the work is complete, changes are submitted to the main branch through a pull request. Each pull-request is reviewed and approved before merging to ensure quality control and alignment between the development reference and the geospatial engine.

Repositories

1. There are two main repositories currently operating:

   - luma-stack: This repository is used to host Luma Geospatial Engine Python package. To contribute to this repository, please refer to `README.md` in this repository for detailed instructions on setting up a developer environment and configuring your local machine.
   - luma-dr: This repository hosts Luma Development Reference. Contributions follow this workflow:
     1. Clone the repository to your local machine.
     2. Create a new branch for your changes.
     3. Edit the relevant `.qmd` files to implement your updates.
     4. Document your modifications in the Braindump under the "Work in Progress" section.
     5. Test your changes by rendering the Quarto book locally to ensure correctness.
     6. Commit and push your changes to your branch.

7. Submit a pull request for review. All changes must be merged via pull request.
8. Proposed changes will be discussed, and once approved, merged into the main branch. Your feature branch will then be deleted.

2. If a new repository is needed (e.g., for an experimental module, plugin, or related service), please consult with an Epistem admin before proceeding.

Branches

1. Every repository should at least have one `main` branch. This branch represents the most up-to-date and stable version of the project.
2. A branch protection is targeted for the `main` branch. This means contributors cannot directly make changes to the `main` branch. Instead, all updates must go through a pull request (PR) process to ensure the code is reviewed and tested before merging. The branch protection rules are as the following:
   - Require a pull-request before merging (please refer to the **Pull-Requests (PRs)** section in this page for more details)
   - Require at least one approval before merging.
3. Each contributor should create a personal working branch from the `main` branch when developing new features or fixes. Once the work is complete, open a pull request to propose merging those changes into the `main` branch, unless there are circumstances where new branch can be created from another feature branch if the work is closely related.
4. Branch naming convention to specify the specific type of a contribution should be further discussed (e.g., `feature/`, `bugfix/`)

Pull-Requests (PRs)

1. All contributions should be submitted through a pull request (PR) from your own branch. A PR is a formal request for your changes to be reviewed and merged into the `main` branch.
2. Keep a PR focused on a specific topic. This specific topic should also correspond to a task in the **Work in Progress** list in Notion. To help identify the task, name the PR title accordingly with what is written in the **Work in Progress** task list.
3. *Only for `luma-stack` and `luma-dr` repository*: when an update in one repository depends on or affects the other, link the related PRs in their descriptions. For example, if a `luma-stack` update requires changes in `luma-dr`, include a link to the corresponding PR in each repository to maintain traceability.

4. As required by the branch protection rules, every PR must receive at least one approval before merging. This approval should come from an Epistem admin. You may also request additional reviewers for feedback.
5. Reviewers should check for correctness, clarity, performance, and proper documentation. Use GitHub's review options:
    - Comment: suggest improvements
    - Request changes: block merge until issues are resolved
    - Approve: accept the PR as ready to merge
6. The PR title should reflect the main change made. This title will be used as the commit message when the PR is merged into the main branch.

Contributor's Roles

1. Only members of the Epistem consortium are authorized contributors to these repositories.
2. There are three different roles set in the epistem-io organization, each with different levels of access:
    - Admin: has full control of repositories (e.g., create/delete repositories, manage access, merge pull requests).
    - Manage: can review and approve contributions, manage issues, and organize repository settings.
    - Write: can create branches, push code, and submit pull requests for review.

    Contributors should know their assigned role before making any changes to ensure proper permissions are in place.

Security and Secrets

Never commit secrets such as passwords, access tokens, or API keys. Add `.gitignore` file to prevent private files from being accidentally uploaded.

Licensing

To be developed. A standardized licensing framework will be provided to ensure consistent open-source and internal usage across repositories.

Releases

To be developed. The release process defines how stable versions of Luma tools and documentation are packaged, tagged, and shared. Releases ensure that users and collaborators can always access a verified version of the software or documentation.

The Development Reference

The development reference is dedicated as a platform for mainly two purposes:

1. **Formalized the decision made in the Braindump.** All discussions' results that were made in the Braindump are documented accordingly, so that all developments made for the Code Implementation are well-tracked.
2. **Facilitate the development of Luma's technical documentation for end-users.** With the use of the development reference as a structured documentation of Luma's development, this documentation can also be a starting point to create Luma's technical documentation for end-users in the future, with reduced effort.

Writing Guidelines for the Development Reference

- The development reference includes several pages, each serving a different thematic purpose in Luma technology
- Each of the pages is written in the following structure:

    1. **Title of the page**
    2. **Input**: a list of input data needed for the related process in the related page. The inputs are divided into:
        - User's Input: list of inputs provided by Luma's users.
        - Input from External Sources: list of inputs received from sources other than Luma's user.
        - Input from Other Modules: list of inputs from other modules in Luma.
    3. **Output**: list of expected output from the process defined in the page.
    4. **Process**: a detailed step-by-step the execution logic of the functionality described on the page. The process should be defined into the following sub-headings:
        - **Step**: a step represents a distinct change in system state or responsibility. A new step must be introduced when:
            * ownership shifts (e.g. from user action to system execution, or between modules), or
            * a new input set is consumed, or
            * an intermediate output is produced that is required by subsequent logic.

  - **Sub-step**: a sub-step represents an atomic action required to complete a step. Sub-steps must not introduce new inputs or outputs beyond those defined for the parent step.
    * **Luma User Journey**: the description of the processes that occur in the user interface. This part is equivalent to the "User Journey" in the Visio workflow diagram.
      1. An error notification should be written in the `callout-important` block type with the title "Error Handling Notification".
      2. A success notification should be written in the `callout-note` block type with the title "Success Notification".
    * **Luma Geospatial Engine**: the description of the processes that occur in the back-end part. This part is equivalent to the "System Response" in the Visio workflow diagram.
      1. The defined function of the related process in the sub-step should be written in the `callout-tip` block type with the title "Related Function".

- Write the Development Reference using clear and easy-to-understand wording that can be understood by audiences with different levels of technical expertise. Avoid technical jargon where possible while still ensuring that all critical elements of the process are clearly and sufficiently described.

> **!** Important
>
> When writing descriptions of a process, make sure to include all the crucial steps that need to be taken in the related process.

Python Method Writing Convention

To be developed

Testing Guideline for Developers

Part I

General Development Settings

Python Environment Setup

1. Overview

Luma stands for Land Use Mapping for All. The geospatial functionalities that powers Luma is called Luma-GE (Luma Geospatial Engine). Luma-GE is shipped as a Python package because it needs to run in more than one context. The same geospatial engine must be importable by an interface layer (for example, a Streamlit-based UI/UX) and also be usable directly from a notebook for development, debugging, and method exploration. Packaging turns "a folder of scripts" into a reliable, versioned engine with a stable import path (`import luma_ge`), repeatable installs across laptops and CI, and a single place to declare runtime requirements.

Luma-GE uses `pyproject.toml` as the source of truth for packaging and installation. This file must live at the repository root. Modern Python tooling (including `pip`) reads `pyproject.toml` to understand the package identity, its required Python version, and the dependencies that must be installed alongside the engine. In practical terms, the `[project]` section is where we define the package name and version, set `requires-python`, and list the dependencies needed by the functions and methods provided by Luma-GE.

> ⚠ Warning
>
> Dependencies will be kept **parsimonious**. Only add a dependency when a function or method genuinely needs it. This keeps installs faster, reduces conflicts, and avoids dragging heavy geospatial binaries into environments that don't need them.

---

2. Goals

The goal is a developer setup that works the same way on different laptops and in CI. Installs should be standard and boring, and a new contributor should be able to get a working environment quickly without any one-off hacks.

---

3. Prerequisites

You need Git installed because `pip` installs from a Git URL by calling `git` under the hood. You also need a Python environment manager. For geospatial stacks, we recommend Miniforge or Mambaforge (Conda-compatible and lightweight) because many geospatial dependencies are easier to install as pre-built binaries than to compile locally.

To confirm Git and Conda/Mamba are available in your shell, run:

```
git --version
conda --version
```

If you are on Windows, avoid adding Python or Conda to your system PATH unless you know exactly why you are doing it. PATH conflicts are a common source of broken environments.

---

4. Output

A working development environment that can run Luma-GE via an interface or via a notebook, and can also build package artefacts when needed.

---

5. Process

Step 1 - Create an isolated environment

Create a dedicated environment for Luma so your system Python stays untouched and dependency conflicts are contained.

```
mamba create -n luma python=3.11
mamba activate luma
```

If you use `conda` instead of `mamba`, the commands are the same (just slower).

Step 2 - Install Luma-GE from GitHub

If you want a quick install of the latest code without cloning the repo, install directly from GitHub. By default we install from the main branch.

```
python -m pip install "luma_ge @ git+https://github.com/epistem-io/luma-stack.git"
```

If you need a specific branch (for example, to test a feature branch or reproduce a bug), pin it explicitly:

```
python -m pip install "luma_ge @ git+https://github.com/epistem-io/luma-stack.git@<branch-nar
```

> **Important**
>
> Installing from a Git URL requires `git` to be installed and available on your PATH.

Step 3 - Quick verification

Confirm the package imports correctly.

```
python -c "import luma_ge; print('luma_ge import OK')"
```

If you want a concrete example of how Luma-GE functions and methods are used in practice (outside the interface layer), review the implementation notebook in the repository:

`notebooks/Module_implementation.ipynb`

```
https://github.com/epistem-io/luma-stack/blob/main/notebooks/Module_implementation.ipynb
```

Step 4 - Tests (coming soon)

> **Warning**
>
> **Coming soon**
> A PyTest suite is planned but not implemented yet.
> When it lands, this section will document the default test command (`pytest -q`), where tests live (`tests/`), naming (`test_*.py`), and recommended patterns (fixtures, `monkeypatch`, and mocking for cloud APIs).

---

6. Change management (three-stream approach)

Python environment changes are high-impact when they change the supported Python version, alter core dependency groups (especially geospatial binaries), modify install commands or extras, or change CI behaviour. These changes should follow the three-stream workflow (Notion → luma-dr → luma-stack) so that the documentation and implementation stay in lock-step.

---

7. References

- UBC Geography (2024) *Getting Started with Conda, Virtual Environments, and Python*. Available at: https://ubc-geography.github.io/computing-resources/development-environments/conda-getting-started.html
- pyOpenSci (2025) *Python Package Structure & Layout*. pyOpenSci Python Package Guide. Available at: https://www.pyopensci.org/python-package-guide/package-structure-code/python-package-structure.html
- PyPA (2025) *VCS Support*. Available at: https://pip.pypa.io/en/stable/topics/vcs-support/
- PyPA (2025) *Writing your pyproject.toml*. Python Packaging User Guide. Available at: https://packaging.python.org/en/latest/guides/writing-pyproject-toml/

Earth Engine Initialization and Authentication

1. Overview

Luma-GE uses the Google Earth Engine (GEE) Python API for cloud-based geospatial processing. Before any module can call Earth Engine, the runtime must be authorised.

In practice, "authorisation" is two things glued together:

First, you authenticate as an identity (a human Google account, or a service account).

Second, Earth Engine needs a Google Cloud project context so quotas and usage accounting are applied to the right place. Recent Earth Engine Python client changes have made this project context effectively mandatory for common authentication modes. If Earth Engine cannot determine a "quota project", requests will fail.

Luma-GE currently supports two working authorisation options, each with clear trade-offs:

1. Manual authentication (interactive OAuth) is best for local development and workflows tied to a human identity. It is interactive (browser + copy/paste code) and is not suitable for unattended deployments.
2. Service account authentication is the default for deployments and CI. It is non-interactive and repeatable, but it has no option for exporting to a user's personal Google Drive.

2. Prerequisites

To use Earth Engine from Python, you need:

A Google account with Earth Engine access, and a Google Cloud project that is registered for Earth Engine and has the Earth Engine API enabled.

If you are using service account mode, you also need a service account created in that Cloud project and a JSON private key for it.

For service accounts, ensure the account has the correct IAM permissions on the project. In practice, the Earth Engine docs point you at Earth Engine resource roles, and some auth modes also require permissions related to Service Usage.

3. Output

An authorised session allowing Luma-GE functions to execute Earth Engine computations, traceable to a specific Google Cloud project.

4. Steps

4a. Manual authentication (interactive OAuth)

Manual authentication is the simplest path for developers working locally. It authenticates you as your own Google account. That matters when you need behaviour tied to a human or organisation identity, such as exporting to your Google Drive.

In practice, you run an auth function, a browser window opens, you sign in with a Google account that has Earth Engine access, and you paste an authorisation code back into the terminal (or a notebook cell). After that, initialise Earth Engine with an explicit project ID so usage is attributed correctly.

In Luma-GE, this is implemented in `luma_ge/ee_config.py:authenticate_manually()`, which calls `ee.Authenticate()` followed by `ee.Initialize(project='YOUR_PROJECT_ID')`.

> **Warning**
>
> Luma-GE does not support running manual authentication inside a Streamlit interface. Earth Engine's interactive flow expects terminal/browser control that a hosted app generally does not have.

4b. Service account authentication (deployment and CI)

A service account is a non-human Google identity intended for machine-to-machine access. This is the recommended mode for deployments, containers, scheduled jobs, and CI pipelines.

To create and download a service account key, use Google Cloud IAM: create a service account under your project, then create a JSON key for it. Treat the JSON key like a password. If it leaks, rotate it.

In Luma-GE, service account auth is implemented in `luma_ge/ee_config.py:initialize_with_service_accou`
The implementation loads the JSON key, sets `GOOGLE_APPLICATION_CREDENTIALS`, and initialises Earth Engine. If you do not pass a project ID explicitly, it attempts to infer it from the JSON (`project_id`).

Where should the key live? Anywhere except Git.

For local development, keep the JSON file outside the repo and point to it using `GOOGLE_APPLICATION_CREDENTIALS`, or store it in a local `auth/` directory that is excluded by `.gitignore`.

For deployments, inject the key via environment variables or a secrets manager. Luma-GE's credential discovery helper (`luma_ge/__init__.py:_find_service_account_file()`) searches in this priority order: `GOOGLE_SERVICE_ACCOUNT_JSON_B64`, then `GOOGLE_SERVICE_ACCOUNT_JSON`, then `GOOGLE_APPLICATION_CREDENTIALS`, before looking for local `auth/` folders and container fallbacks.

> **!** Important
>
> Never commit a service account key to GitHub. Public repos are scraped, history is hard to erase, and forks/caches persist. If a key is exposed, rotate it immediately.

Exports are the main bottleneck with service accounts. Earth Engine supports Drive exports via `Export.image.toDrive`, but Drive exports target the authenticated identity's Drive.

A recent Google Workspace policy change adds a very specific sharp edge. From 15 April 2025, newly created Google Cloud IAM service accounts no longer receive the default Google Workspace storage and cannot own Drive items. In practical terms, a service account created after that date will generally fail to export to "My Drive", because there is no Drive storage space for it to own the exported file.

For service accounts, the recommended pathways are exporting to Google Cloud Storage (GCS) or to an Earth Engine Asset, then sharing/copying results as needed. If you must land results in Drive, a more robust pattern is exporting into a Shared Drive, where the Shared Drive owns the file rather than the service account.

If you want a direct download link for quick debugging or open access downloads (without user login), `ee.Image.getDownloadURL()` is available but intentionally limited (maximum request size 32 MB and maximum grid dimension 10,000 pixels). It will not work for large rasters.

5. Troubleshooting

Most authorisation failures fall into a handful of predictable buckets.

5.1 Authentication fails

If Earth Engine says you are not authenticated, you either have no cached user credentials (manual mode) or the service account key was not found/loaded (service account mode). In Luma-GE, `get_auth_status()` is the quickest sanity check because it performs a trivial request to confirm the session can execute.

If manual authentication suddenly starts failing after a library update (especially in notebook-style flows), check the Earth Engine Python client library release notes. A common error message asks you to grant `roles/serviceusage.serviceUsageConsumer` (or the underlying `serviceusage.services.use` permission) on the relevant project.

If a service account works on one developer laptop but fails on another, it is usually one of these: the key file path is wrong or unreadable; the project is not registered for Earth Engine access; the Earth Engine API is not enabled; the service account does not have the required IAM roles; or permissions have not yet propagated.

5.2 Quota / project errors

If you see an error about a quota project not being set, Earth Engine cannot determine which Cloud project should be charged for requests. The Earth Engine guide recommends setting the project explicitly.

Ensure that (a) the project is registered for Earth Engine access, (b) the Earth Engine API is enabled in Cloud Console, and (c) you initialise with an explicit project in code: `ee.Initialize(project='YOUR_PROJECT_ID')`.

If you are using Application Default Credentials, the Earth Engine guide also suggests running `earthengine set_project YOUR_PROJECT_ID` or `gcloud auth application-default set-quota-project YOUR_PROJECT_ID`.

5.3 Exports fail

Export failures are a common trap. `ee.Image.getDownloadURL()` is intentionally limited to small downloads (maximum request size 32 MB and maximum grid dimension 10,000). If you exceed those limits, switch to batch exports.

For service accounts, it is suggested exporting to Cloud Storage (`Export.image.toCloudStorage`) or to an Earth Engine Asset. For large exports into a user's personal Drive, use manual authentication.

To understand limits and constraints applied to a project (task concurrency, storage, request rates, and other quotas), use the Earth Engine quotas and monitoring documentation, plus Cloud Console quota and monitoring pages.

6. References

Google for Developers (2025) *Authentication and Initialization*. Google Earth Engine Guides. Available at: https://developers.google.com/earth-engine/guides/auth

Google for Developers (2025) *Earth Engine access*. Google Earth Engine Guides. Available at: https://developers.google.com/earth-engine/guides/access

Google for Developers (2025) *Service Accounts*. Google Earth Engine Guides. Available at: https://developers.google.com/earth-engine/guides/service_account

Google for Developers (2024) *ee.Image.getDownloadURL*. Earth Engine API reference. Available at: https://developers.google.com/earth-engine/apidocs/ee-image-getdownloadurl

Google for Developers (2025) *Export.image.toDrive*. Earth Engine API reference. Available at: https://developers.google.com/earth-engine/apidocs/export-image-todrive

Google for Developers (2025) *Export.image.toCloudStorage*. Earth Engine API reference. Available at: https://developers.google.com/earth-engine/apidocs/export-image-tocloudstorage

Google for Developers (2025) *Earth Engine Python client library release notes*. Available at: https://developers.google.com/earth-engine/docs/python-client-lib/release-notes

Google for Developers (2025) *Earth Engine quotas*. Google Earth Engine Guides. Available at: https://developers.google.com/earth-engine/guides/usage

Google for Developers (2025) *Monitoring usage*. Google Earth Engine Guides. Available at: https://developers.google.com/earth-engine/guides/monitoring_usage

Google Cloud (2025) *Set the quota project*. Google Cloud documentation. Available at: https://docs.cloud.google.com/docs/quotas/set-quota-project

Helpers

Part II

Luma Modules

1 Acquisition of Near-Cloud-Free Satellite Imagery

Module Overview

The output of this module is near–cloud-free satellite imagery that corresponds to the user-defined area of interest, which is essential for generating accurate and reliable land use and land cover (LULC) datasets. This module emphasizes the generation of pre-processed and corrected satellite imagery, incorporating procedures such as cloud masking and shadow removal.

Input

Name of Input	Input Type	Details
Area of Interest	User's input	
Target map date	User's input	YYYY or DD-MM-YYYY
Desired spatial resolution	User's input	in meters
Maximum allowable cloud cover	User's input	in percentage
Raw satellite imagery	System's input	
Area of Interest from administrative boundaries	System's input	

Output

1. Selected AOI.
2. Near-cloud-free satellite imagery.

Process

1.1 Selection of AOI

1.1.1 AOI from Shapefile

Luma User Journey

This is a part of **User Journey 1.1: Prerequisite Check**

1. The user can upload their own AOI file in `.zip` format that includes all required shapefile components (`.shp`, `.shx`, `.dbf`, `.prj`). The system emphasizes on all required shapefile components that needs to be put in the `.zip` file. The system also supports uploads of `.kml` or `.kmz` files.
2. The system validates whether the `.zip` file includes all the required shapefile components.

> ! Error Handling Notification
>
> The system provides an error notification if the `.shp` file is not found inside the `.zip` file.

3. The AOI file is saved in a temporary directory, allowing users to go back-and-forth in between modules without having to re-uploading the AOI file.

Luma Geospatial Engine

This sub-step does not involve any operations within the Luma Geospatial Engine.

1.1.2 AOI from Indonesia Administrative Boundaries

In addition to user define upload the system supports the selection of AOI from Indonesia administrative boundaries at regency level.

Luma User Journey

1. The user selects an AOI based on a drop down list of Indonesian regency administrative boundaries
2. The system provides a preview of the selected boundary

Luma Geospatial Engine

1. Load `ee.FeatureCollection` containing Indonesian administrative boundaries from Badan Informasi Geospasial. The data should be uploaded as GEE asset and the link for it must be provided and shared publicly
2. The system extracts the regency name and provide complete list of the available regencies for the user
3. The system convert the `ee.FeatureCollection` to `geodataframe` for visualization, while using the unconverted one as an AOI for the imagery search

> Related Function
>
> `input_utils.load_regency_asset:` Fetching GEE asset containing Indonesia administrative boundaries
> `input_utils.get_regency_name`: Creating a list of regency name for the user
> `input_utils.get_regency_geometry:` Get geometry for the selected regency
> `input_utils.convert_gee_features_to_gdf:` convert `ee.FeatureCollection` to `geodataframe` for visualization in the system

1.2 Correction of Shapefile Geometries

1.2.1 Correction of Shapefile Geometries

Luma User Journey

This is a part of **User Journey 1.1: Prerequisite Check**

1. After the user uploads the AOI, they receive a continuous feedback on each the validation process.

> Success Notification
>
> The system shows a confirmation when the AOI is loaded successfully.

> Error Handling Notification
>
> The system provides an error notification if shapefile validation fails.

2. Once the system succeeded to validate the shapefile input, the system displays the AOI on a map canvas.

Luma Geospatial Engine

This is a part of **System Response 1.1: Area of Interest Definition**

1. The system performs a multi-step geometry sanitization process before converting any shapefile or GeoDataFrame into Earth Engine objects. **These input validation processes for shapefiles are designed mainly for Streamlit compatibility**. Each correction is handled by exactly one dedicated function:

    - The Coordinate Reference System (CRS) is automatically corrected and reprojected to WGS 1984 (EPSG:4326).

    > Related Functions
    >
    > `input_utils.shapefile_validator.fix_crs()`: to resolve coordinate reference system issues.

    - Invalid, empty, or topologically broken geometries are detected and repaired.

    > Related Functions
    >
    > `input_utils.shapefile_validator.clean_geometries()`: to repair invalid geometries.

    - An initial structural validation and basic geometry fixing.

    > Related Functions
    >
    > `input_utils.shapefile_validator.validate_and_fix_geometry()`:to validate and fix geometries issues.

    - Polygon-specific validation, including removal of invalid coordinates and simplification of overly complex shapes.

    > Related Functions
    >
    > `input_utils.shapefile_validator.validate_polygons()`: to checks point geometries, remove invalid coordinates, and simplify polygon geometries.

    - Every individual coordinate is checked to ensure it falls within valid global latitude/longitude ranges.

Related Functions

`input_utils.shapefile_validator.is_valid_coordinate()`: to checks point geometries, remove invalid coordinates, and simplify polygon geometries.

- The number of vertices in each geometry is counted and logged (for diagnostic and simplification decisions).

Related Functions

`input_utils.shapefile_validator.count_vertices()`: to checks point geometries, remove invalid coordinates, and simplify polygon geometries.

- Finally, a comprehensive pre-upload validation is executed to guarantee that every geometry is valid, non-empty, correctly typed, and fully compatible with Earth Engine.

Related Functions

`input_utils.shapefile_validator.final_validation()`: to checks point geometries, remove invalid coordinates, and simplify polygon geometries.

2. After the system succeeded to validate the shapefile, the system converted the GeoDataFrame file format into Earth Engine geometry format.

Related Functions

`input_utils.EE_converter.convert_aoi_gdf()`: to conduct the conversion of `.gdf` input file into `ee.geometry` . Three different conversion approaches was defined in this function:

- Primary method: using `geemap.gdf_to_ee()`
- If primary method fails, proceed to secondary method: manual GeoJSON conversion by union multiple geometries first (`gpd.GeoDataFrame([{'geometry': union_geom}], crs=gdf.crs)`) and convert it to GeoJSON (`gdf.to_json()`). **Only polygon and multipolygon geometry type is supported in this method**.
- If both primary and secondary method fails, proceed to the last option: bounding box conversion to estimate the rectangular approximation of the AOI (`ee.Geometry.Rectangle([bounds[0], bounds[1], bounds[2], bounds[3]])`).

1.3 Selection and Preparation of Satellite Imagery

1.3.1 Satellite Data Acquisition and Filtering

Luma User Journey

This is a part of **User Journey 1.2: Determining Satellite Imageries Parameters**

1. The user is prompted to fill in these parameters for the satellite imagery:
    - Target map date: Choose the range of period of satellite imagery in a single year format (YYYY) or full-date format (YYYY-MM-DD). If YYYY is used as the input format, the year option starts from 1972 until present year. The year 2020 is shown as the default option.
    - Spatial resolution: Specify the desired spatial resolution. During Phase 1, only Landsat data will be utilized; therefore, the spatial resolution will be set on default to 30 meters.
    - Sensor type: a list of sensors available based on the selected target map date input parameter. Landsat 8 OLI is shown to be the default option. A warning message will show about the limited availability of Landsat 1-3 data if the user uses temporal scope input between 1972-1984.

      > **!** Error Handling Notification
      >
      > Since only Landsat data is utilized, the system provides an error notification if no Landsat sensor is available in the selected time range.

    - Maximum cloud cover: Set a preferred cloud cover threshold, or use the default value of 30%.
2. The system provides information on satellite image availability shows the number of total satellite images found based on the user parameter input. The information includes total images found, date range of images, WRS tiles, path row tiles, scene IDs, image acquisition dates, average scene cloud cover, date range, and cloud cover range.

   > **i** Success Notification
   >
   > The system shows the total number of satellite images found.

> **Error Handling Notification**
>
> The system provides an error notification if no satellite images were found based on the user's parameter. The user is prompted to change the cloud cover threshold or change the target map date.

Luma Geospatial Engine

This is a part of **System Response 1.2: Search and Filter Imagery**

1. The system specifies Landsat data collections for optical processing.

   > **Related Functions**
   >
   > `data_acquisition.Reflectance_Data.OPTICAL_DATASETS` defines the properties of Landsat Surface Reflectance (SR) collections (Collection 2, Level-2) to be fetched for optical data processing.

   The system currently supports Landsat 1-3 RAW collection and and Landsat 4-9 surface reflectance (SR) collection. Additionally, Landsat 4 - 9 Top-of-Atmosphere (TOA) is used for retrieving thermal band as additional input band for the classification. This consideration stems from Landsat SR collection thermal band quality which often resulted in no data and prone to over-correction in high contrast land cover feature. Landsat mission availability is as follows:

   - Landsat 1 Multispectral Scanner/MSS (1972 - 1978)
   - Landsat 2 Multispectral Scanner/MSS (1978 - 1982)
   - Landsat 3 Multispectral Scanner/MSS (1978 - 1983)
   - Landsat 4 Thematic Mapper/TM (1982 - 1993)
   - Landsat 5 Thematic Mapper/TM (1984 - 2012)
   - Landsat 7 Enhanced Thematic Mapper Plus/ETM+ (1999 - 2021)
   - Landsat 8 Operational Land Imager/OLI (2013 - present)
   - Landsat 9 Operational Land Imager-2/OLI-2 (2021 - present)

2. The system retrieves images that match user's selected criteria, including spatial resolution, year, and cloud cover based on the following input parameters: AOI, target map date, and cloud cover threshold.

   > **Related Functions**
   >
   > `data_acquisition.Reflactance_Data.get_optical_data()`: to retrieve and the Landsat image collection based on the input parameters. This function uses the `mask_landsat_sr`, `apply_scale_factors`, and `rename_landsat_bands` to return a masked, scaled, and renamed image collection.

- The system performs cloud masking using the quality assurance band (QA_PIXEL) QA_PIXEL band that encodes pixel conditions as bitwise flags using specific bit flags. Deterministic bit tests are applied to identify and exclude pixels affected by cirrus, clouds and cloud shadows. The bit value is adjusted based on the Landsat sensor selected by the user. Landsat 8-9 QA_PIXEL stored per-pixel confidence levels for clouds cirrus, and clouds shadow. These confidence values (ranging from 0–3) are extracted and filtered using hard-coded thresholds, such that pixels with medium to high confidence are masked out. Other Landsat sensor did not have confidence bit, therefore, only deterministic bit are used for cloud masking

  > Related Functions
  >
  > `data_acquisition.Reflactance_Data.mask_landsat_sr()`: Mask clouds, shadows and cirrus for Landsat Collection 2 SR using QA_PIXEL band with confidence threshold

- The system applies scaling to the optical reflectance bands to convert the raw digital numbers (DNs) into physically meaningful surface reflectance values. A scale factor 0.0000275 and offset -0.2 are applied, as defined in the Landsat Collection 2 Surface Reflectance metadata. This conversion standardize the reflectance values to a range of approximately 0 to 1, enabling accurate comparison and analysis across different scenes and sensors.

  > Related Functions
  >
  > `data_acquisition.Reflactance_Data.apply_scale_factors()`: to apply Landsat collection 2 scalling factors using the following formula: Digital Number (DN) * scale_factor + offset. The scale factor will only be applied to optical bands.

- The system renames the bands accordingly with the sensor type input.

  > Related Functions
  >
  > `data_acquisition.Reflactance_Data.rename_landsat_bands()`: to standardize Landsat Surface Reflectance (SR) band names based on sensor type. From 'SR_B*' or 'B*' to 'NIR', 'GREEN', etc.

3. The system then use the same parameter as multispectral data and use it to search collection 2 TOA data, except for Landsat 1-3 MSS. The TOA data thermal band (namely, band 10) is then stacked with multispectral band from SR data. Since band 11, contain larger calibration uncertainty, only band 10 is used. No additional pre-processing is conducted for thermal band since digital number value alone is considered satisfactory

for discriminating land cover features.

> Related Functions
>
> `data_acquisition.Reflactance_Data.get_thermal_bands()`: to retrieve and the Landsat image collection based on the input parameters.

4. The system evaluates whether the retrieved imagery meets the required spatial resolution, input year, and cloud cover thresholds. If suitable images are found, the system generates a composite by calculating the median value across the selected imagery, as implemented using the `median()` function from Earth Engine. The system also then clips and mosaics the satellite imagery.

1.4 Mosaic and Composite for Satellite Imagery

Luma Geospatial Engine

This is a part of **System Response 1.2: Search and Filter Imagery**

The final image (single layer) which will be presented to the user is created using `final_image` methods, which resulted in mosaic imagery or temporal aggregation composite imagery.

1. Mosaicking

   Mosaicking approach simply stacks all images in the collection and, for each pixel, selects the value from the topmost image according to the collection's ordering. The system implement quality mosaic, which allows the use of quality band that affected the order of mosaic

   > Related Functions
   >
   > `data_acquisition.final_Image.get_quality_mosaic` create a quality mosaic image based on quality band (source band). `add_quality_band` compute or select the band that will be used for creating the mosaic. Three quality band is supported, namely NDVI and NIR. The mosaic is created based on the highest pixel value of the corresponding band

2. Temporal Aggregation

   Temporal aggregation used statistical reducers, (such as median) to combine pixel values from an image collection over time. A **median composite** often produces a noticeably cleaner image because cloud and other high-reflectance features tend to occupy the upper tail of the distribution. This approach, naturally excludes the feature, resulting in fewer artifacts and a more consistent surface reflectance. However, if at certain location there's

no valid pixel across the image collection, the location will resulted in no-data (blank region). Temporal aggregation function is paired with `calculate_coverage` to provide report of the valid pixels in an AOI.

> Related Functions
>
> `data_acquisition.final_Image.get_temporal_composite:` Create a temporal aggregation composite using earth engine reducers. Supported reducers are `median`, `mean`, `min`, `max`, and `percentile`

3. Valid Pixel Reporting

   To provide comprehensive report on the final image creation, system provide the data range which composite is created. Additionally, valid pixel and no data is estimated and reported to the user.

> Related Functions
>
> `calculate_coverage` estimate, the valid pixels (unmasked by cloud masking) within AOI. This quantified the 'no data' or blank blocks on the final image. This served as alternative approach in reporting the cloud cover within AOI, since in theory, the cloud cover within AOI is already masked out during cloud masking procedure. This function is used within the `get_temporal_composite` and reported in the final image creation.
> The `calculate_coverage` serves as an alternative to calculate cloud cover within AOI. Since cloud cover should be masked out during cloud masking and remove during temporal aggregation

1.5 Visualization and Saving of Processed Satellite Imagery

1.5.1 Visualization and Saving Processed Satellite Imagery

Luma User Journey

This is a part of **User Journey 1.3: Download Satellite Imagery**

1. The system visualize the satellite imagery on a canvas map in bands NIR (near-Infrared), RED, and GREEN. The map is centralized on the AOI.
2. The user can download the satellite imagery through two options:

a. Direct download: Downloads the image locally in the user's device. This option is only available if the satellite image is less than 32 MB. Direct download settings are defined from Streamlit.

> **Success Notification**
>
> The system shows a confirmation if the URL for direct download of satellite imagery is successfully made.

> **Error Handling Notification**
>
> The system provides an error notification if the system fails to create the URL for direct download and suggests the user to use Google Cloud Storage instead or use smaller AOI area.

b. Export to Google Cloud Storage: Downloads the image to Google Cloud Storage.

> **Success Notification**
>
> The system shows a confirmation if the satellite image is successfully saved to Google Cloud Storage.

> **Error Handling Notification**
>
> The system provides an error notification if the system fails to export the image to Google Cloud Storage.

3. The user is given the option to continue to Module 2

> **Error Handling Notification**
>
> The system disables the option to continue to the next module if the system does not detect a satellite imagery saved inside the system.

Luma Geospatial Engine

This is a part of **System Response 1.3: Download Satellite Imagery and Continue to Next Module**

1. The system collects the statistics of the satellite images found.

> Related Functions
>
> `data_acquisition.Reflactance_Stats.get_collection_statistics()`: to get comprehensive statistics about the gathered image collection.

2. For details on exporting to Google Cloud Storage, refer to Earth Engine Initialization and Authentication page.
3. The system saved the satellite imagery as an object to be loaded in the next modules.

2 LULC Classification Scheme

Module Overview

This module prompt the user to define the list of land cover or land use class which they're going to classified using the Luma platform. Luma supports user-defined classification schemes and default classification or templates for commonly used classification schemes.

Input

Name of input	Input Type	Details
User's LULC list of classes	User's input	Entered with `.csv` file or manually defined on the UI
Other existing LULC list of classes	System's input	RESTORE+ LULC class

Output

1. LULC list of classes to be applied in the final classification result

Process

> ❗ Error Handling Notification
>
> This module cannot be accessed if the system does not detect any saved satellite imagery from Module 1.

2.1 Selection of Classification Scheme

Luma User Journey

This is a part of **User Journey 2.1: Choosing a classification scheme**

1. The user is given the options of creating the classification scheme:
    a. Upload the classification scheme
    b. Manually define the scheme on the UI
    c. Use an existing classification scheme

Luma Geospatial Engine

This sub-step does not involve any operations within the Luma Geospatial Engine.

- If the user chooses to upload their own schema, the system proceeds to Section 2.1.1
- If the user chooses to define the classification manually on the UI, the system proceeds to Section 2.1.2
- If the user chooses a default classification scheme, the system proceeds to Section 2.1.3

2.1.1 User Uploading Their Own Classification Scheme

Luma User Journey

1. The user is prompted to upload the `.csv` file containing the classification scheme information.
2. The system displays a guidance notification indicating that the uploaded `.csv` file must contain at least three columns: a class ID (numeric identifier), a class name, and an optional hex color code specifying the display color for each class.

3. The system automatically detects column headers based on common naming conventions. If the expected headers are not found, the user must manually assign the correct column names.
4. The user is prompted to finalize the scheme by clicking a button.

> **Success Notification**
>
> The system shows a confirmation if the classification table is successfully saved to the system.

> **Error Handling Notification**
>
> The system shows an error message if:
>
> - The class ID is not a valid number
> - There is a duplicate class ID

Luma Geospatial Engine

This is a part of **System Response 2.1.a: Uploading classification scheme**

1. The system automatically detects column headers based on common naming conventions for ID, class name, and color code. If the expected headers are not found, the user must manually assign the correct column names.

> **Related Functions**
>
> `classification_scheme.LULC_scheme_Manager.auto_detect_csv_columns()`: this function automatically detects the column headers. The set for the common naming conventions are currently hardcoded as the following:
>
> - For ID: “id”, “classid”, “kode”, “code”
> - For class name: “classname”, “class”, “kelas”, “name”, “nama”
> - For color code: “color”, “colorcode”, “warna”, “colour”, “hex”, “palette”

2. The system validates the uploaded `.csv` to ensure IDs are numeric and unique, applies a set of distinct colors if no color code was provided by the user. The user can change the color code manually.

> **Related Functions**
>
> `classification_scheme.LULC_scheme_Manager.process_csv_upload()`: to validate class data by checking ID integrity, assigning colors

(`_generate_distinct_colors`), and returning a success status with a summary message.
`classification_scheme.LULC_scheme_Manager._generate_distinct_colors()`: to generate a predefined set of distinct colors for LULC classes (up to 20). If the number of classes exceeds this limit, the method falls back to generating random colors via `_generate_random_colors()`.
`classification_scheme.LULC_scheme_Manager._generate_random_colors()`: to generate random hex color code.
`classification_scheme.LULC_scheme_Manager.finalize_csv_upload()`: to finalize the CSV file by applying any user-assigned colors, saving and sorting the class list, and clearing temporary data.

2.1.2 User Entering the Classification Manually

Luma User Journey

1. The user is prompted to fill out a form for each class ID, class name, and hex color code.
2. The system stores each class entry individually as it is added.
3. All added classes are displayed in a summary table for preview.
4. The user can edit or delete the classes in the summary table preview.
5. The user is prompted to finalize the scheme by clicking a button.

> **Success Notification**
>
> The system shows a confirmation if the classification table is successfully saved to the system.

Luma Geospatial Engine

This is a part of **System Response 2.1.b: Manually define classification scheme**

1. The system validates user-provided class input for numeric ID, non-empty class name, and uniqueness (if adding new classes).

> **Related Functions**
>
> `classification_scheme.LULC_scheme_Manager.validate_class_input()`: Validates class ID and name, checks for uniqueness, and returns a validity flag with an error message if invalid.
> `classification_scheme.LULC_scheme_Manager.add_class()`: Adds a new class

or updates an existing class, validates inputs, assigns colors, sorts classes, and updates the next available ID.

2. The system normalizes class name inputs through a sequential process: it first applies Unicode normalization using the NFKD form, converts all characters to lowercase, replaces common separators (_, -) with spaces, removes non-alphanumeric characters, collapses multiple consecutive spaces, and applies light lexical normalization to reduce superficial grammatical differences, such as singular versus plural forms (e.g. "forest" and "forests"). Missing values (`None` or `NA`) are returned as None. Optionally, words within the class name can be sorted alphabetically to enable order-independent matching (e.g. "Forest Mixed" = "Mixed Forest")
3. Temporary edit mode flags are managed to differentiate between adding and updating classes.

> Related Functions
>
> `classification_scheme.LULC_scheme_Manager.edit_class()`
> `classification_scheme.LULC_scheme_Manager.delete_class()`
> `classification_scheme.LULC_scheme_Manager.cancel_edit()`

2.1.3 User Selecting a Default Classification Scheme

Luma User Journey

1. The user is prompted to select a default classification scheme. In the current development, only classification scheme from RESTORE+ is available.
2. The user is given the option to choose only a specific subset of the default classes, rather than being required to utilize the entire set.
3. The user is prompted to finalize the scheme by clicking a button.

> Success Notification
>
> The system shows a confirmation if the classification table is successfully saved to the system.

Luma Geospatial Engine

This is a part of **Response System 2.1.c: Using default classification scheme**

1. The system loads the chosen classification scheme classes.

> Related Functions
>
> `classification_scheme.LULC_scheme_Manager.load_default_scheme()`: to load an existing classification scheme from a dataset.
> `classification_scheme.LULC_scheme_Manager.get_default_schemes()`: stores the RESTORE+ LULC class in a dictionary.

2. If the user selects a subset of the default classes, the system stores the selection into a variable that will be used for the reclassification process in Module 6.

> Related Functions
>
> `classification_scheme.LULC_scheme_Manager.store_classes_of_interest()`: to store the user's class of interest as a variable that will be used for the reclassification process in Module 6.

2.2 Saving and Downloading the LULC Classification Table

2.2.1 Saving and Downloading the LULC Classification Table

Luma User Journey

This is a part of **User Journey 2.2: Saving and Downloading the LULC Classification Table**

1. The user can optionally download the table in `.csv` file format and save it in their local device.
2. The system gets a confirmation of the number of classes saved in the system.
3. The user is given the option to proceed to Module 3.

> Error Handling Notification
>
> The system disables the option to continue to the next module if the system does not detect a satellite imagery saved inside the system.

Luma Geospatial Engine

This is a part of **System Response 2.2: Save LULC Classification Table and Continue to Next Module**

1. The system generates a `.csv` file containing the final LULC classification scheme table.

> Related Functions
>
> `classification_scheme.LULC_scheme_Manager.get_csv_data()` : to generate a `.csv` file containing the final classification table that has been standardized by `get_dataframe`.
> `classification_scheme.LULC_scheme_Manager.get_dataframe()` : to standardize the column names of the LULC classification table to "ID", "Land Cover Class", "Color Palette"

2. The system checks whether the user has defined at least one class.

> Related Functions
>
> `classification_scheme.LULC_scheme_Manager.has_classes()`: to check if any classes are defined.
> `classification_scheme.LULC_scheme_Manager.get_class_count()`: get the information of the number of the defined classes.

3. The system saved the LULC class classification scheme as an object to be loaded in the next modules.

3 Sample Data Generation

Module Overview

To be developed.

Input

Name of input	Input Type	Details
User's sample data	User's Input	
Default sample data	System's Input	RESTORE+ sample data
LULC classification class table	Input from Other Modules	Output from Module 2
AOI	Input from Other Modules	Output from Module 1
Near-cloud free satellite imagery	Input from Other Modules	Output from Module 1

Output

1. Statistical analyses of the sample data.

Process

3.1 Checking Prerequisites from Previous Modules

3.1.1 Checking Prerequisites from Previous Modules

Luma User Journey

This is a part of **System Response 3.1: Checking prerequisites from previous modules**

1. If the user has not completed Module 1 and Module 2, the system displays a warning message prompting them to finish those modules first.

   > ! Error Handling Notification
   >
   > This module cannot be accessed if the system is missing the required inputs from the Input from Other Modules.

2. The user is prompted to choose how to add sample data, either by uploading their own dataset, using on-screen sampling, or using default sample data .

Luma Geospatial Engine

This sub-step does not involve any operations within the Luma Geospatial Engine.

3.2 Sample Data Generation

3.2.1 User Uploads Their Own Sample Data

Luma User Journey

This is a part of **User Journey 3.1: Identifying sample data availability**

1. The user is prompted to upload a `.zip` file containing their sample data.

> **Error Handling Notification**
>
> The system provides an error notification if the `.shp` file is not found inside the `.zip` file.

2. The user is prompted to specify the column header that specify the LULC class. This is a part of **User Journey 3.2a: Uploading sample data**

Luma Geospatial Engine

1. The system verifies the format data input of the `.shp` inside the `.zip` . This is a part of **System Response 3.2.a: Verifying the uploaded sample data format**
2. Once the input format data is validated, the system runs Section 3.2.2
3. The system loads the sample data after verifying the upload
    - Check the initial sample count and conduct sample filtering based on its geometry, making sure all of the sample is located inside the area of interest
    - Handle a large dataset by stratified sampling so that the sample does not exceed 5000 features, and maintain class distribution representation
    - Convert the shapefile into a geodataframe
    - Updates the class field identifier in the training data dictionary.

> **Related Function**
>
> `sample_data.SyncTrainData.LoadTrainData()`: to load the sample data into the system by conducting several checks

> **Feature note**
>
> The main issue is that the original stratified sampling algorithm made sequential `.getInfo()` calls for each class label when calculating class sizes and extracting features, which could cause browser crashes or exceed Earth Engine quotas when processing large datasets (5000+ features). The improved implementation consolidates all server-side computations into a single aggregated request, keeping all operations on the server side until the final `.getInfo()` call, thus avoiding excessive client-server communication and quota limitations.

3.2.2 Verifying Sample Data from User's Input

Luma User Journey

1. The user receives continuous feedback on the status of their sample data verification. Five steps are displayed in the status panel:

    1. Checking the field containing class names
    2. Validating the class entries
    3. Checking whether class IDs are sufficient
    4. Verifying that sample points fall within the AOI
    5. Generating a summary of the sample data status

2. The user is shown a preview of the uploaded sample data on a map canvas and a preview table of the uploaded sample data.

3. The user is then prompted to process the sample data.

    > **i** Success Notification
    >
    > The system shows a confirmation if the sample data is successfully verified.

    > **!** Error Handling Notification
    >
    > The system provides an error notification if the system fails to conduct the sample data verification, along with the specific message at which step the validation failed.

4. The user is provided with the summary display of the sample data.

    > **i** Success Notification
    >
    > A notification shows a highlight of which class has insufficient sample data, the number of sample data that falls outside the AOI, and the total number of classes that has sample data outside the AOI.

5. The system provides an option to move to on-screen sampling to add more sample data. This is a part of **User Journey 3.2.b: Choosing to add new sample data for insufficient class**

6. If the user is satisfied with the sample data, the user is prompted to finalize the sample data by clicking a button.

    > **i** Success Notification
    >
    > The system shows a confirmation if the sample data is successfully saved into the system.

Luma Geospatial Engine

This is a part of **System Response 3.2.b: Verifying the uploaded sample data's content and adding new sample data**

1. The system checks which column in the sample dataset should be used as the class label.

   > Related Functions
   >
   > `sample_data.SyncTrainData.SetClassField()`: Set the class name field for sample data.

2. The system validates class labels in the sample dataset by filtering out class values that do not exist in the classification scheme from Module 2, and updates the sample data and validation report accordingly.

   > Related Functions
   >
   > `sample_data.SyncTrainData.ValidClass()`: Validate classes in sample data.

3. The system checks the number of sample data for each class. The minimum required sample size per class is currently hard-coded at 20.

   > Related Functions
   >
   > `sample_data.SyncTrainData.CheckSufficiency()`: Check if there are sufficient samples per class.

4. The system filters the training samples so only those that spatially overlap the AOI remain, updates validation counts, and records warnings if AOI filtering fails.

   > Related Functions
   >
   > `sample_data.SyncTrainData.FilterTrainAoi()`: Check if there are sufficient samples per class.

5. The system generates a summary table of the user's uploaded sample data. The summary includes:

   - LULC class ID
   - LULC class name
   - Number of sample data of the LULC class
   - Percentage of the sample data count
   - Sufficiency category of the LULC class' sample data

> **Related Functions**
>
> `sample_data.SyncTrainData.TrainDataRaw()`: Check if there are sufficient sample data per class.

3.2.3 On-Screen Sampling

Luma User Journey

This is a part of **System Response 3.3.a: On screen sampling facility**

1. The system shows an interactive map where user can add a sample data. If the user already uploaded their own sample data, the verified sample data will be shown on the interactive map.

   > **Success Notification**
   >
   > The system shows a confirmation that the uploaded sample data has been loaded to the on-screen canvas, along with the number of sample data uploaded.

2. The system lists all LULC classes defined in Module 2. The user assign new sample points accordingly to the correct class.
3. The system offers an option to save the result from on-screen sampling process to their local device.
4. The user is prompted to finalize the on-screen sampling process by clicking a button.

   > **Success Notification**
   >
   > Once, finalized the system shows a summary of the number of the modified sample data after on-screen process.

5. The system then moves to Section 3.3.

Luma Geospatial Engine

This sub-step does not involve any operations within the Luma Geospatial Engine. The on-screen sampling feature was developed using `folium` for the Streamlit development.

3.2.4 Using Default Sample Data

Luma User Journey

1. The user is automatically directed to use the default sample data if the user chose to use an existing LULC scheme in Module 2
2. The system displays the area of the AOI and the number of sample data loaded on a map canvas.

> **!** Error Handling Notification
>
> The system shows an error notification if RESTORE+ data is failed to be loaded. This could be due to the AOI located outside the scope of RESTORE+ dataset, or the AOI is too big.

> **i** Success Notification
>
> The system displayed a summary of the number of sample data loaded from RESTORE+ data that matches with the AOI saved in the system, and the area of AOI in km2.

Luma Geospatial Engine

1. The system retrieves an existing sample data accordingly with the chosen LULC scheme. In this development phase, the RESTORE+ sample data is retrieved from Google Earth Engine asset.

3.3 Sample Data Preview

3.3.1 Sample Data Preview

Luma User Journey

This is a part of **System Response 3.4: Verified sample data preview**

1. The user is provided with the summary display of the **updated** sample data.

> **i** Success Notification
>
> A notification shows a highlight of which class has insufficient sample data, the number of sample data that falls outside the AOI, and the total number of classes that has sample data outside the AOI.

2. The user is prompted to confirm that they will use the sample dataset.

> **Success Notification**
>
> The system shows a confirmation that the sample data has been saved into the system.

3. The user is given the option to proceed to Chapter 4 .

> **Error Handling Notification**
>
> The system disables the option to continue to the next module if the system does not detect any sample data stored in the system.

Luma Geospatial Engine

1. The system runs Section 3.2.2 for the **updated** sample data.
2. The system saved the sample data as an object to be loaded in the next modules.

4 Sample Data Quality Analysis

Module Overview

This module assesses the spectral separability of LULC classes using sample data and near-cloud-free satellite imagery. Pairwise class separability is quantified using the Transformed Divergence (TD) method based on pixel-level spectral statistics, using only the spectral data from Module 1.

This module is diagnostic only and does not produce inputs for subsequent modules. Its outputs support interpretation of class confusion and provide justification for feature enhancement in later stages of the workflow.

Input

Name of Input	Input Type	Details
Parameters for the separability analysis	User's input	Contains: • Spatial resolution of the imagery from Module 1 can be adjusted for the separability analysis • Number of maximum pixels which the analysis will run for each pair of classes
Sample dataset	Input from Other Modules	Output from Module 3
Near-cloud free satellite imagery	Input from Other Modules	Output from Module 1

Output

1. Separability analysis result.
2. Spectral profile visualization and text description.

Process

> ❗ Error Handling Notification
>
> This module cannot be accessed if the system is missing the required inputs from the Input from Other Modules.

4.1 Specifying the Separability Analysis Parameter

4.1.1 Specifying the Separability Analysis Parameter

Luma User Journey

This is a part of **User Journey 4.1: Determining separability analysis parameter**

1. The user is prompted to fill in the parameters for separability analysis. The system provides a default value for each of the parameters as the following:
    - Spatial resolution used for the separability analysis: 30 meter
    - Maximum number of pixels which the separability analysis will be conducted to: 5000 for each class, to prevent crash in the analysis.
2. The system shows a message showing Transformed Divergence method will be used to conduct the analysis. In this development phase, there is no option for the user to change the separability analysis method.

Luma Geospatial Engine

This sub-step does not involve any operations within the Luma Geospatial Engine.

4.2 Conducting the Separability Analysis

4.2.1 Initialize the Separability Analysis

This is a part of **System Response 4.1: Separability Analysis Workflow**

Luma User Journey

1. The user is prompted to run the separability analysis

Luma Geospatial Engine

This sub-step does not involve any operations within the Luma Geospatial Engine.

4.2.2 Validating the sample data's geometry

Luma User Journey

1. The system shows a progress update indicating that the separability analysis is currently at Step 1 of 7.

Luma Geospatial Engine

1. The system validates the geometry of the sample data stored in the system from Module 3 output.

> Related Function
>
> `input_utils.shapefile_validator.validate_and_fix_geometry()`:to validate and fix geometries issues.

> Error Handling Notification
>
> An error message is displayed if the system failed to run `validate_and_fix_geometry()`.

4.2.3 Converting sample data into Earth Engine format

Luma User Journey

1. The system shows a progress update indicating that the separability analysis is currently at Step 2 of 7

Luma Geospatial Engine

1. The system converts sample data from `.gdf` data into Earth Engine Feature Collection format

> Related Function
>
> `input_utils.shapefile_validator.convert_roi_gdf()`: to convert geo-dataframe into EE Feture Collection. The supported multi-geometries type for this conversions are MultiPoint and MultiPolygon.

> Error Handling Notification
>
> An error message is displayed if the system failed to run `convert_roi_gdf()`.

4.2.4 Starting the separability analysis computation

Luma User Journey

1. The system shows a progress update indicating that the separability analysis is currently at Step 3 of 7

Luma Geospatial Engine

1. The system initializes the Related Function's for separability analysis with the appropriate required input objects for the `class`.

> Related Function
>
> `sample_data_quality.sample_quality()`: a `class` to handle all the workflows for separability analysis.

4.2.5 Counting sample data statistics

Luma User Journey

1. The system shows a progress update indicating that the separability analysis is currently at Step 4 of 7.

Luma Geospatial Engine

1. The system retrieves the basic statistic information of the sample data.

> Related Function
>
> `sample_data_quality.sample_quality.sample_stats()` : to retrieve the sample data's total sample count, unique classes, class-wise sample counts, and class balance,

and includes class names when provided.
`sample_data_quality.sample_quality.get_sample_stats_df()` : to turn the statistic information into a dataframe format.

4.2.6 Extracting spectral value

Luma User Journey

1. The system shows a progress update indicating that the separability analysis is currently at Step 5 of 7. The system also highlights that this step will take some time.

Luma Geospatial Engine

1. The system extracts the pixel-level spectral values for each sample data by sampling the composite image at the specified scale. The system also limits the maximum number of pixels per LULC class that will be extracted to avoid memory overload.

> Related Function
>
> `sample_data_quality.sample_quality.extract_spectral_values()` : to extract and compile spectral reflectance values for all sample data. The result is a dataframe containing band values for each sample data.
> `sample_data_quality.sample_quality.limit_samples_per_class()` : limits (down-samples) the number of sample data per class whenever the number of available samples exceeds the maximum pixel threshold with random sampling.

4.2.7 Calculating the extracted spectral feature statistics

Luma User Journey

1. The system shows a progress update indicating that the separability analysis is currently at Step 6 of 7.

Luma Geospatial Engine

1. The system computes detailed pixel-level statistics for each LULC class (mean, standard deviation, minimum, maximum, median, and sample count) based on the extracted spectral values and formats these results into a structured summary table.

> Related Function
>
> `sample_data_quality.sample_quality.sample_pixel_stats()` : calculates per-class spectral statistics (mean, std, min, max, median, count) for all bands.
> `sample_data_quality.sample_quality.get_sample_pixel_stats_df()` : converts the computed statistics into a dataframe.

4.2.8 Calculating separability index

Luma User Journey

1. The system shows a progress update indicating that the separability analysis is currently at Step 7 of 7.

> Success Notification
>
> The system shows a confirmation that separability analysis has been successfully conducted.

Luma Geospatial Engine

1. The system computes the class-to-class separability using Transformed Divergence (TD).
2. The system generates a pairwise separability matrix and converts the results into a structured dataframe.
3. The system categorizes the number of good, weak, and poor class pairs as a summary for user.

> Related Function
>
> `sample_data_quality.sample_quality.check_class_separability()`: calculates the pairwise separability values between classes using Transformed Divergence. The choice of using TD as the algorithm is still hard-coded.
> `sample_data_quality.sample_quality.get_separability_df()`: converts separability results into a dataframe with class names and interpretation levels.
> `sample_data_quality.sample_quality.lowest_separability()`: extracts the class pairs with the lowest separability scores.
> `sample_data_quality.sample_quality.sum_separability()`: produces a summary report counting how many class pairs fall under good, weak, or poor separability categories.

4.3 Visualization of Reference Data's Spectral Profile

4.3.1 Visualization of Reference Data's Spectral Profile

This is a part of **User Journey 4.2: Analyzing separability index result and visualizing sample data**

Luma User Journey

1. The user is shown a summary of the number and proportion of sample data for each LULC class from the output of Section 4.2.5.

   > **i** Success Notification
   >
   > The system displays a summary of the number and proportion of sample data for each LULC class in a tabular format.

2. The user is shown a summary of the extracted sample features for each LULC class from the output of Section 4.2.7.

   > **i** Success Notification
   >
   > The system displays a summary of the extracted sample features for each LULC class in a tabular format.

3. The user is presented with an overall summary of the separability analysis based on the output of Section 4.2.8.

   > **i** Success Notification
   >
   > The system displays a highlight of separability category (good, weak, or poor) for each class, and highlights which LULC classes require improvement. A dataframe containing details of the pairwise separability is also displayed.

4. The user is provided with interactive visualizations to explore spectral distributions and clustering. Available visualization options include histograms, boxplots, and scatter plots.

   > **i** Success Notification
   >
   > The system displays visualization options: histograms, boxplots, 3D scatter plot, and 2D scatter plots.

Luma Geospatial Engine

1. The system provides four different visualization options of the spectral distributions and clustering.

> Related Functions
>
> `sample_data_quality.spectral_plotter.plot_histogram()`: generates overlaid interactive histograms of selected bands showing class-wise frequency distributions
> `sample_data_quality.spectral_plotter.plot_boxplot()`: creates interactive boxplots summarizing per-class band statistics (median, quartiles, outliers) for selected bands.
> `sample_data_quality.spectral_plotter.interactive_scatter_plot()`: produces an interactive 2D scatter plot of two bands with class coloring and hover details for exploring class clustering.
> `sample_data_quality.spectral_plotter.static_scatter_plot()`: renders a static 2D scatter plot with optional confidence ellipses
> `sample_data_quality.spectral_plotter.add_elipse()`: draws a confidence ellipse axis for a class' 2D band distribution (used by `static_scatter_plot`).
> `sample_data_quality.spectral_plotter.scatter_plot_3d()`: builds an interactive 3D scatter plot to explore three-band feature space and rotate/zoom clusters.

5 Predictor Introduction

> Under development
>
> This module's backend is not available yet. This module is not available in Phase 1.

6 LULC Map Generation

Module Overview

This module performs supervised Random Forest classification to generate a land cover map. It includes input review, hyperparameter definition, feature extraction, model training, hard/soft classification, feature importance, and accuracy evaluation using test data.

Input

Name of input	Input type	Details
Sample data split proportion	User's input	Optional, if not set then use default value
Random Forest (RF) parameters	User's input	Optional, if not set then use default value
Satellite imagery within the AOI	Input from other modules	Module 1
LULC classes	Input from other modules	Module 2, including the selection of the classes of interest if the user uses the default classification scheme.
Sample data	Input from other modules	Module 3

Output

1. Trained LULC classification model.
2. Model accuracy report if validation data available (including overall accuracy, precision, recall, F1-score, Kappa, model error matrix). This output is only available if the user has a validation data from the splitted the sample data
3. Classified LULC raster map.

Process

> ! Error Handling Notification
>
> This module cannot be accessed if the system is missing the required inputs from the Input from Other Modules

6.1 Checking Prerequisites from Previous Modules

This is a part of **System Response 6.1: Reviewing input specifications for model training**

6.1.1 Checking Prerequisites from Previous Modules

Luma User Journey

1. The user is displayed a verification of the required inputs stored in the system.

Luma Geospatial Engine

1. The system validates availability of the required inputs.

6.2 Extracting spectral feature from sample data

6.2.1 Splitting the sample data

Luma User Journey

1. The user defines the proportion to split the sample data into training and validation dataset.
2. If the user does not split the sample data, the system shows a warning that all the sample data will be used as training data.

3. The user is required to set feature extraction parameter, including class label column and the pixel size.

> **Success Notification**
>
> The system shows a confirmation that feature extraction has been completed, and the summary of the number of training data and validation data split.

Luma Geospatial Engine

1. The system conducts feature extraction for both training and validation data accordingly to the proportion set by the user.
2. The system uses stratified random split method to conduct the data splitting. This method randomly selects samples within each class to proportionally create training and testing sets while preserving class distribution.

> **Related Functions**
>
> `classification.FeatureExtraction.stratified_split()` : performs stratified random split of sample data based on LULC class labels.

6.3 Creating classification model

6.3.1 Creating classification model

Luma User Journey

This is a part of **User Journey 6.2: Determining classification's specification**

1. The user is given the option to use default value of the RF parameters. The default value are as the following:
   - Number of trees: 50, 100, or 150 are given as options
   - Variables per split: square root of total spectral band features used
   - Minimum leaf population: 1
2. The user is given the option to manually configure the classification parameter:
   - Number of trees
   - Variables per split: square root of total spectral band features used, or can adjust manually
   - Minimum leaf population

3. The user is prompted to verify the inputs and run the process to create the classification model.

> **Error Handling Notification**
>
> The process to create a classification model is disabled if the user has not extracted the spectral feature

> **Success Notification**
>
> The system shows a confirmation that a classification model has been created.

Luma Geospatial Engine

This is a part of **System Response 6.2: Classification process**

1. The system create the classification model using hard classification method.

> **Related Functions**
>
> `classification.Generate_LULC.hard_classification()` : trains a RF classifier (`smileRandomForest`) using extracted training data and applies it to the input imagery to produce a hard (pixel-level) multiclass.

6.3.2 Reviewing classification model's quality

Luma User Journey

This is a part of **User Journey 6.3: Review classification result**

1. The user is shown a feature-importance graph displaying each variable used to train the classification model. In this development phase, only spectral bands are used as classification variables.
2. The user is prompted to begin the process of calculating the model's accuracy assessment.

> **Success Notification**
>
> The system shows a confirmation of the completion of model accuracy assessment.

3. The user is shown the results of the model's accuracy assessment, presented in the following structure:

- Overall model accuracy (overall accuracy percentage, kappa coefficient, average F1-score, and G-Mean score)
- Class-level accuracy (producer's accuracy, user's accuracy, F1-score, and G-Mean score), also visualized in a confusion matrix with an accompanying highlight summary of the LULC classes with the lowest accuracy.

Luma Geospatial Engine

This is a part of **System Response 6.3: Testing model's accuracy**

1. The system extracts feature importance from the trained Random Forest model and presents a ranked table of bands (or a fallback equal-weight estimate if true importance is unavailable).

   > Related Functions
   >
   > `classification.Generate_LULC.get_feature_importance()` : extracts and returns model feature importance as a dataframe; if the model explanation is not available, falls back to an equal-weight estimate.

2. The system calculates model evaluation metrics on the validation dataset. This option only available if the user perform data split.

   > Related Functions
   >
   > `classification.Generate_LULC.evaluate_model()` :classifies the validation data with the trained classifier, builds the confusion matrix, and returns a dictionary of accuracy metrics

3. Confusion matrix and model accuracy summary are then rendered in the user-interface.

6.3.3 Visualizing LULC classification map result

Luma User Journey

1. The user can visualizes the LULC classification result on a canvas map along with the training data layer.
2. The user is given the option to adjust the class color visualization, or to reset it accordingly with the saved determined color saved in Module 2.
3. If the user selected a subset of classes of the default classes in Module 2, the map will only show the classes of the user's interest. The rest of the default classes that are not selected will be shown as "Others".

Luma Geospatial Engine

1. The system loads the user's class of interest stored from Module 2.
2. The system reclassifies the default classification scheme's map into a map that only shows the user's class of interest and an additional class labeled as "Others".

> Related Functions
>
> `classification.Generate_LULC.reclassify_map_by_classes()`: reclassifies the default classification scheme final map to only show the user's class of interest and show the other classes as "Others".

6.3.4 Downloading the LULC classification map result

Luma User Journey

1. This step is optional for the user.
2. The user can download the classified LULC map through two options:
   a. Direct download: Downloads the image locally in the user's device. This option is only available if the classification map result is less than 32 MB.

> Success Notification
>
> The system shows a confirmation if the URL for direct download of classified LULC map is successfully made.

> Error Handling Notification
>
> The system provides an error notification if the system fails to create the URL for direct download and suggests the user to use Google Cloud Storage instead.

   b. Export to Google Cloud Storage: Downloads the image to Google Cloud Storage.

> Success Notification
>
> The system shows a confirmation if the classified LULC map is successfully saved to Google Cloud Storage.

> **!** Error Handling Notification
>
> The system provides an error notification if the system fails to export the classified LULC map to Google Cloud Storage.

Luma Geospatial Engine

1. This sub-step does not involve any operations from Luma Geospatial Engine. Direct download settings are defined from Streamlit, using the same workflow as defined in Section 1.5.
2. For details on exporting to Google Cloud Storage, refer to Earth Engine Initialization and Authentication page.

7 Thematic Accuracy Assessment

Module Overview

This module performs a comprehensive thematic accuracy assessment of the land cover map using ground reference data. It includes data verification, upload/validation of test data, accuracy matrix calculation (OA, Kappa, PA, UA), confidence intervals, F1-scores, and formatted summary reports for display.

Input

Name of input	Input type	Details
Validation map	User's input	In shapefile format.
Classified LULC map	Input from Other Modules	Module 6

Output

- Confusion matrix.
- Thematic accuracy metrics: Overall Accuracy (OA), Kappa Coefficient, Producer's Accuracy (PA), User's Accuracy (UA).

Process

7.1 Checking Prerequisites from Previous Modules

This is a part of **System Response 7.1: Verification data**

7.1.1 Checking Prerequisites from Previous Modules

Luma User Journey

1. The user is displayed a verification of the required inputs stored in the system.

> ! Error Handling Notification
>
> This module cannot be accessed if the system is missing the required inputs from the Input from Other Modules.

Luma Geospatial Engine

1. The system validates availability of the required inputs.

7.2 Thematic Accuracy Assessment

7.2.1 Uploading the Validation Map

Luma User Journey

This is a part of **User Jorney 7.2 Upload data**

1. The user is prompted to upload the validation map in a `.zip` file. The user is reminded that the validation map should have the same ID class as the one used for the classification.

> i Success Notification
>
> The system shows a confirmation that the validation data has been validated.

> **Error Handling Notification**
>
> The system shows an error message if the uploaded validation map fails the validation process.

2. The system displays the uploaded validation map on a canvas map along with the tabular data.

Luma Geospatial Engine

1. The system verifies if the `.shp` of the validation map is the uploaded inside the `.zip` file.

> **Error Handling Notification**
>
> The system provides an error notification if the `.shp` file is not found inside the `.zip` file.

2. The system conducts geometry fixes on the uploaded validation map.

> **Related Function**
>
> `input_utils.shapefile_validator.validate_and_fix_geometry()`:to validate and fix geometries issues.

> **Error Handling Notification**
>
> An error message is displayed if the system failed to run `validate_and_fix_geometry()`.

3. The system converts the validation map from `.gdf` data into Earth Engine Feature Collection format

> **Related Function**
>
> `input_utils.shapefile_validator.convert_roi_gdf()`: to convert geo-dataframe into EE Feture Collection. The supported multi-geometries type for this conversions are MultiPoint and MultiPolygon.

> **Error Handling Notification**
>
> An error message is displayed if the system failed to run `convert_roi_gdf()`.

7.2.2 Starting the thematic accuracy assessment

Luma User Journey

This is a part of **User Journey 7.3: Thematic accuracy assessment**

1. The user is prompted to fill in the parameters for the thematic accuracy assessment, including specifying the column that refers to the LULC ID class and the pixel size of the classified LULC map.
2. The user can optionally set the confidence interval for the thematic accuracy assessment.
3. The user is prompted to start the thematic accuracy assessment process.

> **!** Error Handling Notification
>
> The system shows an error message if the system fails to run the thematic accuracy assessment.

Luma Geospatial Engine

This is a part of **System Response 7.3: Thematic accuracy assessment**

1. The system performs a thematic accuracy assessment by validating inputs, sampling the classified LULC map at reference points.

> Related Functions
>
> `thematic_assessment.Thematic_Accuracy_Assessment.validate_inputs()`: checks whether the classified LULC map, validation map, LULC ID class field, and pixel size parameter meet the required conditions before the assessment runs. This function is used in `run_accuracy_assessment()`.
> `thematic_assessment.Thematic_Accuracy_Assessment._extract_confusion_matrix_data()`: extracts overall accuracy, kappa, per-class accuracies, and the confusion matrix array from an `ee.ConfusionMatrix` object. This function is used in `run_accuracy_assessment()`.
> `thematic_assessment.Thematic_Accuracy_Assessment._calculate_confidence_interval()`: Computes the confidence interval for overall accuracy using a normal approximation based on the number of correct and total samples. This function is used in `run_accuracy_assessment()`.
> `thematic_assessment.Thematic_Accuracy_Assessment._calculate_f1_scores()`: Calculates the F1 score for each class using the producer's and user's accuracy values. This function is used in `run_accuracy_assessment()`.
> `thematic_assessment.Thematic_Accuracy_Assessment.run_accuracy_assessment()`: Executes the full thematic accuracy workflow, including validation, sampling

(`sampleRegions()`), metric extraction (`errorMatrix()`), confidence interval calculation, and compilation of final results.

7.2.3 Reviewing the thematic accuracy result

Luma User Journey

This is a part of **User Journey 7.4: Preview thematic accuracy result**

1. The system displays the overall accuracy metrics result:
    - **Overall Accuracy (OA)**: Percentage of correctly classified validation samples
    - **Kappa Coefficient**: Agreement beyond chance, accounting for class distribution
    - **Confidence interval at 95%**: The lower and upper bounds that describe the uncertainty range of the overall accuracy estimate at the 95% confidence level
    - **Validation map number**: The total number of validation samples used in the thematic accuracy assessment
2. The system displays the accuracy metrics result on class-level:
    - **Producer's Accuracy**: Measure of classification completeness (1 - omission error)
    - **User's Accuracy**: Measure of classification reliability (1 - commission error)
    - **F1 score**: Summarizes Producer's and User's Accuracy metrics
3. The system displays the confusion matrix.
4. The user is offered the option to download the overall accuracy metrics result summary or the class-level accuracy result in a `.csv` format.
5. The user is prompted to return to Module 6 to improve the model classification quality.

Luma Geospatial Engine

This sub-step does not involve any operations from Luma Geospatial Engine.

8 Post Classification Analysis

> Under development
>
> This module's backend is not available yet.